PHYSICAL ORGANIZATION

File organizations

Outline

0. Introduction

1. Data on External Storage

2. Files and records.

3. Operations on files.

4. File organization

5. Unordered files

6. Ordered files

0. Introduction

- The basic abstraction of data in a DBMS is a collection of *records*, or *file*, and each file consists of one or more *blocks*.
- A **file organization** is a method of arranging the records in a file when the file is stored on disk.
- Each file organization makes *certain operations efficient* but *other operations expensive*.
- Techniques called *access methods* (*indexing* and *external hashing*) can help when we have to access a collection of records in multiple ways.

1. Data on external storage (1)

- A DBMS stores *vast quantities of data*, and the data must *persist across program executions*.

- Data are stored on external storage devices such as *disks* or *tapes*. They are fetched into main memory for processing, and written back to disks for persistent storage.

- Data are transferred between disk and main memory for processing and storage in units of a **block**. Its typical values range from 512B, to 8KB.

- The **cost of block I/O** (input from disk to main memory, and output from memory to disk) *dominates* the *cost of typical DB operations*.
  - DBMS are carefully optimized to *minimize the I/O cost*.

1. Data on external storage (2)

- *Disks* are the most important external storage devices.
  - They allow us to retrieve any block at a (more or less) fixed cost per block.
  - Further, if we read several blocks in the order that they are stored physically, the cost can be much less than the cost of reading the same pages in a random order.

- *Tapes* are sequential access devices and force us to read data one block after the other.
  - They are mostly used to archive data that is not needed on a regular basis.

2. Files and records

- A **record** consists of a collection of data values, or items, where each value is formed of one or more bytes and corresponds to a particular **field** of the record.
- A **data type** associated with each field specifies the type of values the field can take. A collection of field names and their corresponding data types constitutes a **record format**.

- A **file of records** is a sequence of records that may reside on several blocks.
- Each record has a unique identifier called a **record id** (or **rid** for short). The rid allow us to identify the disk address of the block that contains the record.

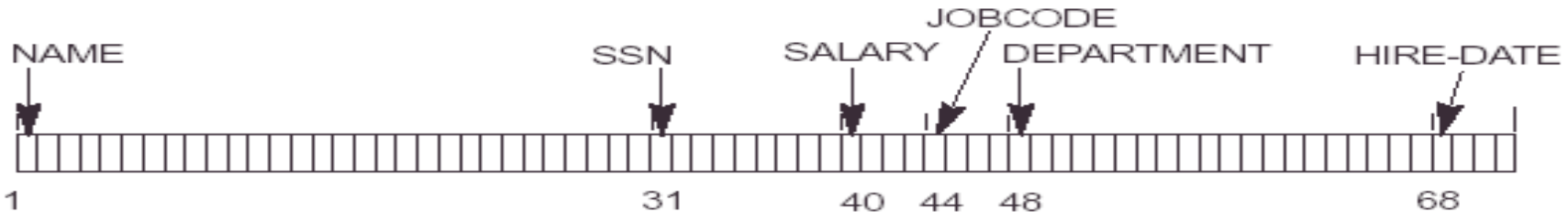
- We can think of a block as a collection of **slots**, each of which contains one record.
  A *record id* is usually the pair *<block id, slot number>*.

2.1. Types of records (1)

- A file can have **fixed-length records** or **variable-length records**.

- In a file of fixed-length records, all records have *the same format*.

  Example:
  A fixed length record with 6 fields and size of 71 bytes.



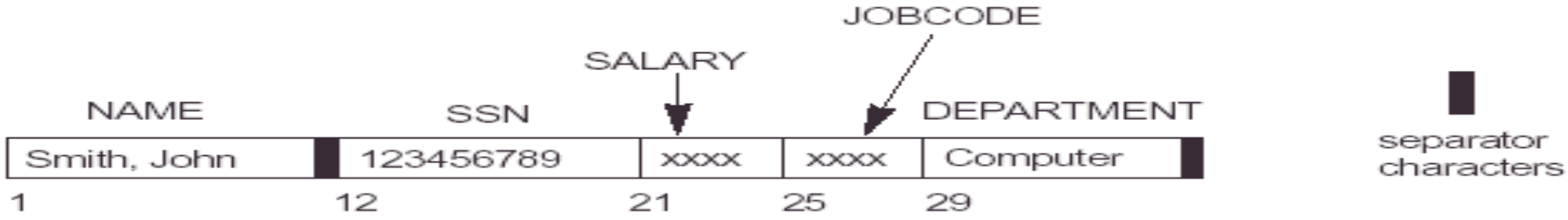
- Files of variable-length records require additional information to be stored in each record, such as *separator characters* and *field types*.

2.1. Types of records (2)

- A file may have variable length records for various reasons:

– The file records are of the same format, but one or more of the fields are of varying size (**variable-length fields**).

Example:



Here NAME and DEPARTMENT fields are variable length fields and a special separator character is used to terminate these fields.

2.1. Types of records (3)

– The file records are of the same format, but one or more of the fields may have multiple values for individual records (**repeating field**).

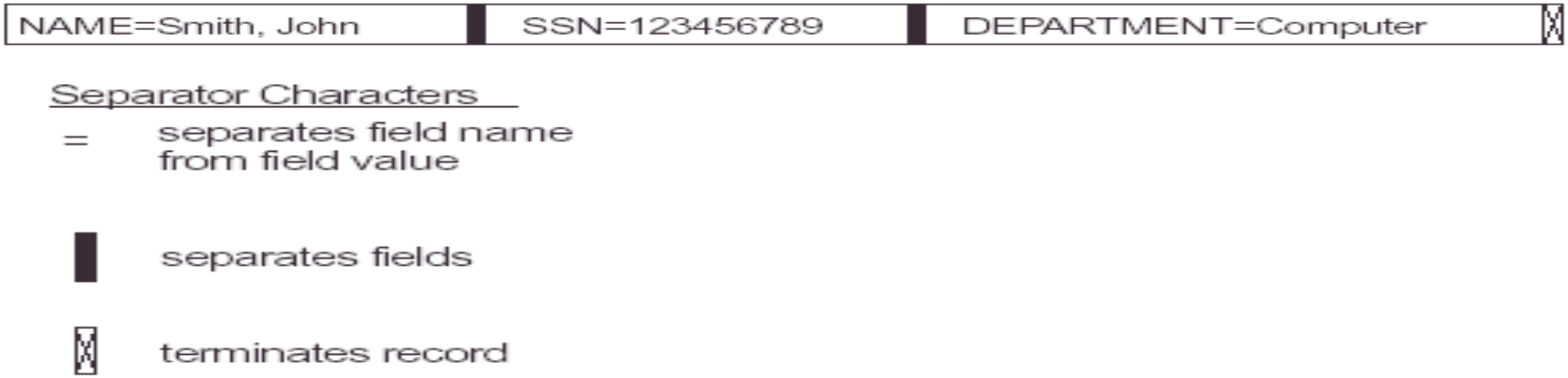
  To represent a repeating field we need one separator character to separate repeating values and one separator character to terminate the record.

2.1. Types of records (4)

– The file records are of the same format, but one or more of the fields are optional (**optional fields**).

One solution for representing these records is to include in each record field names with the actual field values.

Example:



2.1. Types of records (5)

– The file records are of varying formats and hence of varying sizes (**mixed file)**.

  To represent such files, each record is preceded by a *record type* indicator.

- It is possible to represent a file that physically would have variable length records as a fixed-length records file. This technique results in *waste of space*.

- Programs that process files with variable length records are *more complex* that those for fixed-length records.

2.2. Record blocking (1)

- Records are stored on disk blocks. The **blocking factor bfr** for a file is the (average) number of file records stored in a disk block.

- Let B be the *block size in bytes*, and R be the *size in bytes* of the fixed-length records of a file. Then we can fit $bfr = \lfloor B/R \rfloor$ records per block.

  $\lfloor x \rfloor$ is the *floor function*.

- If R does not divide B exactly, we have some *unused space in each block* equal to

  $B - (bfr*R)$ bytes

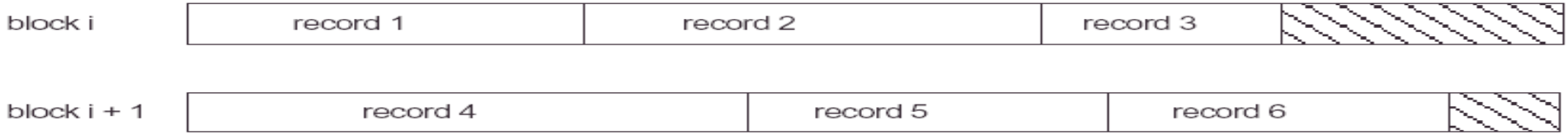
- The *number of blocks* b needed for a file of r records is $b = \lceil r / bfr \rceil$.

  $\lceil x \rceil$ is the *ceiling function*.

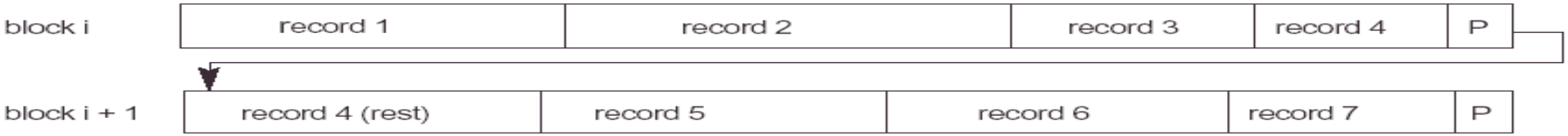
2.2. Record blocking (2)

- File records can be **unspanned** (no record can span two blocks) or **spanned** (a record can be stored in more than one block).

- Example:



Unspanned record organization



Spanned record organization

- Usually, in a file of fixed-length record, *unspanned blocking is used,* while *spanned blocking* is used with files of variable-length records.

2.3. File block allocation on disk

- The physical disk blocks that are allocated to hold the records of a file can be **contiguous**, **linked**, or **indexed**.

- In *contiguous allocation* the file blocks are allocated to *consecutive disk blocks*.

- In *linked allocation* each file block contains a *pointer to the next file block*. A combination of contiguous and linked allocation is also possible by creating and linking *clusters* of consecutive disk blocks.

- In the *indexed allocation*, one or more *index blocks contain pointers* to the actual file blocks.

2.4. File headers

- A **file descriptor** (or **file header**) includes information that describes the file, such as the *field names* and their *data types*, and the *addresses of the file blocks* on disk.

3. Operations on files (1)

- Typical operations on files are grouped into **retrieval operations** and **update operations**.

- <u>Retrieval operations</u>

– **OPEN:** Prepares the file for access, and associates a pointer that will refer to a *current* file record at each point in time.

– **FIND:** Searches for the first file record that satisfies a certain condition, and makes it the current file record.

– **FINDNEXT:** Searches for the next file record (from the current record) that satisfies a certain condition, and makes it the current file record.

– **READ:** Reads the current file record into a program variable.

– **CLOSE**: Terminates access to the file.

3. Operations on files (2)

Update operations

INSERT: Inserts a new record into the file, and makes it the current file record.

DELETE: Removes the current file record from the file, *usually by marking the record* to indicate that it is no longer valid.

MODIFY: Changes the values of some fields of the current file record.

3. Operations on files (3)

- In addition to the previous **record-at-a-time** operations, additional **set-at-a time** higher level operations may be applied to a file.

- **REORGANIZE**: Reorganizes the file records. For example, the records marked deleted are physically removed from the file or a new organization of the file records is created.

- **READ_ORDERED**: Read the file blocks in order of a specific field of the file.

- Some files may be **static,** meaning that update operations are rarely performed; other, more **dynamic** files may change frequently, so update operations are constantly applied to them.

4. File organization (1)

- A **file organization** is a way of arranging the records in a file when the file is stored on disk.

- A successful file organization should perform as efficiently as possible the operation we expect to apply frequently to the file. Often we are interested in supporting more than one operation.

- Example:
  Sorting a file of employee records on the name field makes it easy to retrieve employees in alphabetical order, but we may also want to retrieve all employees who earn $40,000.

4. File organization (2)

- Unfortunately, there may not be an organization that allows all needed operations on a file to be implemented efficiently.

- In such cases, a *compromise must be chosen* that takes into account the expected importance of retrieval and update operations.

  Indexes are built to speed up operations that are not efficiently supported by the basic organization of records in a file.

5. Unordered Files (1)

- Also called a *heap* or a *pile* file.

- New records are inserted at the end of the file.

- To search for a record, a *linear search* through the file records is necessary.
  - This requires reading and searching half the file blocks (b/2) on the average *if only one record satisfies the search condition*.
  - If *no record* or *several records* satisfy the search condition, the program must read and search all b blocks in the file.
  - Hence, *searching for a record is quite expensive*.

- Record insertion is quite efficient: the last disk block is copied into the buffer; the new record is added; and the block is rewritten back to the disk. The address of the last block is kept in the file header.

5. Unordered Files (2)

- A common technique for record deletion is to have an extra bit, called deletion marker, stored with each record.

  A record is deleted by setting the deletion marker to a certain value.

  A different value of the marker indicates a valid (not deleted) record.

  Search programs consider only valid records in a block when conducting their search.

  This technique requires *periodic reorganization* of the file to acclaim for the unused space of deleted records.

- Reading the records of unordered file in order of a particular field requires creating a sorted copy of the file.

6. Ordered Files (1)

- File records are kept sorted by the values of an *ordering field*. If the ordering field is also a key field of the file, it is also called *ordering key*.

- An ordered file is also called a *sequential file*.

- Example

  Suppose that NAME is the ordering key field of a file.

→

6. Ordered Files (2)

	NAME	SSN	BIRTHDATE	JOB	SALARY	SEX
block 1	Aaron, Ed					
	Abbott, Diane					
			⋮			
	Acosta, Marc					
block 2	Adams, John					
	Adams, Robin					
			⋮			
	Akers, Jan					
block 3	Alexander, Ed					
	Alfred, Bob					
			⋮			
	Allen, Sam					
block 4	Allen, Troy					
	Anders, Keith					
			⋮			
	Anderson, Rob					
block 5	Anderson, Zach					
	Angell, Joe					
			⋮			
	Archer, Sue					
block 6	Arnold, Mack					
	Arnold, Steven					
			⋮			
	Atkins, Timothy					
			⋮			
block n –1	Wong, James					
	Wood, Donald					
			⋮			
	Woods, Manny					
block n	Wright, Pam					
	Wyatt, Charles					
			⋮			
	Zimmer, Byron					

6. Ordered Files (3)

- Insertion is expensive: records must be inserted in the *correct order*. It is common to keep a separate unordered *overflow* (or *transaction*) file for new records to improve insertion efficiency; this is periodically merged with the main ordered file.

- A *binary search* can be used to search for a record on its *ordering field value*. This requires reading and searching $\lceil \log_2 b \rceil$ file blocks, whether the record is found or not. This is an important improvement over linear search.

- Reading the records in order of the ordering field is quite efficient.

- Ordered files with an additional access method called primary index are called **indexed-sequential files**.